Problema. Sistem de transport public

Sarcină pentru studiul individual 2

Elaborați o aplicație Java pentru managementul transportului public dintr-un oraș.
Programul trebuie să permită evidența mijloacelor de transport public, afișarea vehiculelor disponibile, calcularea venitului total generat într-o zi și salvarea / încărcarea datelor din fișier.
În sistem vor exista următoarele tipuri de mijloace de transport:

- Autobuz
- Tramvai
- Metrou

Scopul aplicației

Să se realizeze un program orientat pe obiect, prin care se gestionează mijloacele de transport public ale unui oraș, folosind moștenirea, polimorfismul, colecțiile și operațiile cu fișiere.

Cerințe

1. Creați clasa de bază `TransportPublic`

Clasa `TransportPublic` va descrie informațiile generale despre orice mijloc de transport public.

Câmpuri:

- `numarVehicul`
- `numarLocuri`
- `traseu`
- `tarif`

Explicație:

- `numarVehicul` – identificatorul vehiculului
- `numarLocuri` – numărul total de locuri disponibile
- `traseu` – traseul pe care circulă vehiculul
- `tarif` – costul unei călătorii

Metode:

- constructor fără parametri
- constructor cu parametri
- metode `get` și `set`
- metoda `toString()`
- metoda `citireDate()`
- metoda `afisareDate()`
- metoda `calculeazaVenitZilnic(int numarCalatori)`

---

2. Creați clasele derivate

Creați următoarele clase derivate din `TransportPublic`:

- `Autobuz`
- `Tramvai`
- `Metrou`

---

3. Clasa `Autobuz`

Clasa `Autobuz` va fi derivată din `TransportPublic`.

Câmp suplimentar recomandat:

- `tipCombustibil`

Exemple:

- diesel
- electric
- hybrid

Metode:

- constructori
- metode `get` și `set`
- supradefinirea metodei `toString()`
- supradefinirea metodei `citireDate()`
- supradefinirea metodei `afisareDate()`
- supradefinirea metodei `calculeazaVenitZilnic(...)`, dacă doriți să particularizați calculul

---

4. Clasa `Tramvai`

Clasa `Tramvai` va fi derivată din `TransportPublic`.

Câmp suplimentar recomandat:

- `numarVagoane`

Metode:

- constructori
- metode `get` și `set`
- supradefinirea metodei `toString()`
- supradefinirea metodei `citireDate()`
- supradefinirea metodei `afisareDate()`
- supradefinirea metodei `calculeazaVenitZilnic(...)`, dacă este necesar

---

5. Clasa `Metrou`

Clasa `Metrou` va fi derivată din `TransportPublic`.

Câmp suplimentar recomandat:

- `numarStatii`

Metode:

- constructori
- metode `get` și `set`
- supradefinirea metodei `toString()`
- supradefinirea metodei `citireDate()`
- supradefinirea metodei `afisareDate()`
- supradefinirea metodei `calculeazaVenitZilnic(...)`, dacă este necesar

---

6. Fiecare clasă trebuie să conțină metode de acces

Pentru toate câmpurile din clase, trebuie să existe:

- metode `get`
- metode `set`

Aceste metode vor permite accesarea și modificarea controlată a datelor.

---

7. Utilizați moștenirea și polimorfismul

Toate mijloacele de transport vor fi stocate într-o colecție de tipul clasei de bază:

```
ArrayList<TransportPublic> vehicule
```

În această colecție se vor adăuga obiecte de tip:

- `Autobuz`

- `Tramvai`
- `Metrou`

La parcurgerea colecției, metodele:

- `afisareDate()`
- `calculeazaVenitZilnic(...)`

vor fi apelate polimorf.

---

8. Creați clasa `SistemTransport`

Clasa `SistemTransport` va gestiona toate mijloacele de transport.

Câmp recomandat:

```
ArrayList<TransportPublic> vehicule
```

Metode recomandate:

- `adaugaVehicul(TransportPublic v)`
- `afiseazaToateVehiculele()`
- `afiseazaVehiculeDisponibile()`
- `calculeazaVenitTotalPeZi()`
- `salveazaInFisier()`
- `incarcaDinFisier()`

Aici apare relația de **agregare**, deoarece sistemul gestionează mai multe vehicule de transport public.

---

9. Scrieți clasa `Test` / `Main`

În clasa `Test` sau `Main`, realizați următoarele operații:

a) Instanțiați câteva obiecte

- creați mai multe obiecte de tip `Autobuz`
- creați mai multe obiecte de tip `Tramvai`
- creați mai multe obiecte de tip `Metrou`

b) Adăugați vehiculele în sistem

- introduceți toate obiectele într-o colecție

c) Afișați vehiculele disponibile

- afișați datele tuturor mijloacelor de transport din sistem

d) Calculați venitul zilnic pentru fiecare vehicul

Pentru fiecare vehicul, utilizatorul poate introduce:

- numărul de călători dintr-o zi

iar programul va calcula:

```
venit = numarCalatori * tarif
```

e) Calculați venitul total generat pe zi

- adunați veniturile tuturor vehiculelor
- afișați suma totală

f) Salvați datele în fișier

- salvați vehiculele într-un fișier

g) Încărcați datele din fișier

- citiți datele din fișier și afișați-le

---

10. Meniu minimalizat

Programul trebuie să conțină un meniu simplu, prin care utilizatorul să poată alege operațiile dorite.

Exemplu de meniu:

1. Adaugă vehicul

2. Afișează toate vehiculele
3. Afișează vehiculele disponibile
4. Calculează venitul zilnic pentru un vehicul
5. Calculează venitul total pe zi
6. Salvează datele în fișier
7. Încarcă datele din fișier
8. Ieșire

---

11. Operații cu fișiere

Programul trebuie să permită:

- salvarea datelor despre vehicule într-un fișier
- încărcarea datelor din fișier

Datele pot fi salvate:

- într-un fișier text
  sau
- într-un fișier binar

Elevul poate alege varianta dorită, dar programul trebuie să demonstreze clar operațiile de scriere și citire.

---

12. Cerințe de implementare

La realizarea aplicației trebuie să:

- folosiți principiile POO
- aplicați moștenirea între clase
- utilizați constructori supraîncărcați
- redefiniți metoda `toString()`
- utilizați o colecție pentru stocarea vehiculelor
- implementați metode pentru afișare, calcul și lucru cu fișiere
- respectați identarea și claritatea codului

Rezultat așteptat

Programul trebuie să permită:

- evidența mijloacelor de transport public
- afișarea vehiculelor disponibile
- calcularea venitului zilnic pentru fiecare vehicul
- calcularea venitului total generat pe zi
- salvarea datelor în fișier
- încărcarea datelor din fișier

Indicații pentru elevi

Pentru o implementare corectă:

- porniți de la clasa de bază `TransportPublic`
- creați apoi clasele derivate `Autobuz`, `Tramvai` și `Metrou`
- implementați clar metodele de afișare și calcul
- stocați toate vehiculele într-o singură colecție
- testați programul cu mai multe obiecte din fiecare tip
- demonstrați clar salvarea și citirea datelor din fișier